



Object Oriented Programming - Python

Chepter II - Functions

Mr. Joydeep Patar

Assistant Professor
Department of Computer Engineering
Dayananda Sagar University

September 8, 2025

Table of Contents

- 1 Introduction
- 2 Advantages of using Function
- 3 Function Definition
- 4 Function Definition: Example
- 5 Calling a Function
- 6 Examples

What is Function?

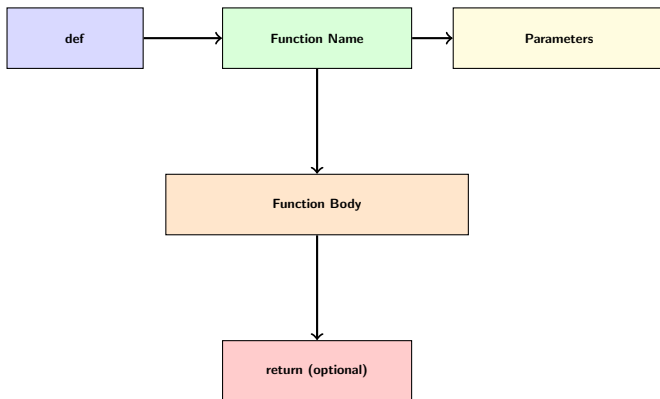
A function is a block of reusable code that performs a specific task.

Types of Functions

- Built-in Function – `print()`, `len()`, `max()` etc.
- User-defined Function

- **Code Reusability**
 - Write once and reuse the same code wherever required.
- **Modularity**
 - Breaks a large program into smaller, manageable parts.
- **Improved Readability**
 - Function names and structures make code easier to read.
- **Easy Testing and Debugging**
 - Functions can be tested individually to isolate issues.
- **Avoids Repetition**
 - Reduces code duplication, saving development time.

Function Definition in Python



Syntax:

Python Function Syntax

```
def function_name(parameters):  
    # function body  
    return value (optional)
```

Colon starts function body

Python Code

```
def add_numbers(a, b):      # Function definition starts with 'def'
    result = a + b          # Function body
    return result           # Return statement (optional)
```

- `def` → Keyword to define a function
- `add_numbers` → Function name
- `a, b` → Parameters
- `:` → Indicates the start of function body
- `return result` → Returns the output of the function

Key Point

A function runs only when it is called. It does ****not**** execute on its own.

Syntax:

- `function_name()`
- `function_name(arg1, arg2, ...)`

Examples:

- `add()`
- `add(20)`
- `add(10.56, 34)`

What is the Output?

Example: 1

```
def check(x):  
    if x % 2 == 0:  
        return "Even"  
    else:  
        return "Odd"
```

Question: What will be the output of `check(13)`?

Options:

- (A) Even
- (B) Odd
- (C) Error
- (D) None

What is the Output?

Example: 2

```
def square(x):  
    return x * x  
def add_one(y):  
    return y + 1
```

Question: What will be the output of the expression `square(add_one(3))`?

Options:

- (A) 9
- (B) 10
- (C) 16
- (D) 25

What is the Output?

Example: 3

```
def check_sum_prime(a, b):  
    def is_prime(n):  
        if n < 2:  
            return False  
        for i in range(2, int(n**0.5) + 1):  
            if n % i == 0:  
                return False  
        return True  
    total = a + b  
    print("Sum:", total)  
    if is_prime(total):  
        print("It is Prime")  
    else:  
        print("It is Not Prime")  
check_sum_prime(8, 5)
```

What is the output?

- (A) Sum: 13
It is Prime
- (B) Sum: 13
It is Not Prime
- (C) Sum: 12
It is Prime

Different types of return statements

```
1 def add_sub(y):  
2     x = 10  
3     c = x + y  
4     d = y - x  
5     return c, d  
6  
7 sum, sub = add_sub(20)  
8 print(sum)  
9 print(sub)
```

• 1. No Return Value

- Returns None by default.

```
1 def say_hello():  
2     print("Hello")
```

• 2. Return Single Value

```
1 def square(x):  
2     return x * x
```

• 3. Return Expression

```
1 def calc(x, y, z):  
2     return x * y + z
```

• 4. Return Multiple Values (Tuple)

```
1 def operations(a, b):  
2     return a + b, a - b
```

• 5. Return List / Dict / Set

```
1 def get_students():  
2     return ["Alice", "Bob"]  
3  
4 def get_profile():  
5     return {"name": "John", "age": 22}
```

• 6. Conditional Return

```
1 def sign(x):  
2     if x > 0:  
3         return "Positive"  
4     else:  
5         return "Non-positive"
```

• 7. Return Function (Closure)

```
1 def outer():  
2     def inner():  
3         return "Inner"  
4     return inner
```

• 8. Return Lambda

```
1 def multiplier(n):  
2     return lambda x: x * n
```

• 9. Return Object

```
1 class Student:  
2     pass  
3  
4 def get_student():  
5     return Student()
```

Nested Functions in Python

- A **nested function** is a function defined inside another function.
- It can access variables from the enclosing (outer) function.

Example:

```
1 def outer(name):  
2     def inner():  
3         print("Hello, ", name)  
4     inner()  
5  
6 outer("Joydeep")
```

Output:

Hello, Joydeep

Passing a Function as a Parameter

- In Python, functions are first-class objects.
- This means functions can be passed as arguments to other functions.
- Useful for callbacks, higher-order functions, and functional programming.

Example:

```
1 def greet(name):  
2     return "Hello, " + name  
3  
4 def call_function(func, value):  
5     result = func(value)  
6     print(result)  
7  
8 call_function(greet, "Joydeep")
```

Output: Hello, Joydeep

Function Returning Another Function

- In Python, functions can return other functions.
- This allows for powerful design patterns like closures and decorators.

Example:

```
1 def outer(msg):  
2     def inner():  
3         print("Message:", msg)  
4     return inner  
5  
6 my_func = outer("Hello from Joydeep")  
7 my_func()
```

Output: Message: Hello from Joydeep

Actual and Formal Parameters

- **Formal Parameters:** Variables defined in the function definition that accept the values passed.
- **Actual Parameters:** Real values or variables passed to the function when it is called.
- **Used for:** Data transfer between function call and definition.

Example:

```
1 def greet(name):          # 'name' is a formal parameter
2     print("Hello, ", name)
3
4 greet("Joydeep")          # "Joydeep" is an actual parameter
```

Output: Hello, Joydeep

- Positional Arguments
- Keyword Arguments
- Default Arguments
- Variable Length Arguments
- Keyword Variable Length Arguments

Positional Arguments (with Integers)

- Arguments are passed in the order defined in the function.

Example:

```
1 def multiply(a, b):  
2     print("Product:", a * b)  
3  
4 multiply(5, 3)
```

Output: Product: 15

- Arguments passed using parameter names.

Example:

```
1 def subtract(a, b):  
2     print("Result:", a - b)  
3  
4 subtract(b=10, a=25)
```

Output: Result: 15

- If no value is passed, the default is used.

Example:

```
1 def power(base, exponent=2):  
2     print("Power:", base ** exponent)  
3  
4 power(5)           # uses default exponent = 2  
5 power(2, 3)        # overrides default
```

Output:

- Power: 25
- Power: 8

Variable Length Arguments (*args)

- Accepts multiple integers as positional arguments.

Example:

```
1 def total_sum(*nums):  
2     print("Sum:", sum(nums))  
3  
4 total_sum(5, 10, 15)
```

Output: Sum: 30

Keyword Variable Length Arguments (**kwargs)

- Accepts any number of integer values with keys.

Example:

```
1 def marks(**scores):  
2     for subject, mark in scores.items():  
3         print(f"{subject}: {mark}")  
4  
5 marks(Math=95, English=88, Science=92)
```

Output:

- Math: 95
- English: 88
- Science: 92

- ❶ Create a function in Python.
- ❷ Create a function with variable length of arguments.
- ❸ Return multiple values from a function.
- ❹ Create a function with a default argument.
- ❺ Create an inner function to calculate the addition in the following way.
- ❻ Create a recursive function.
- ❼ Assign a different name to a function and call it through the new name.

Local and Global Variables

Definition:

- **Global Variable:** Declared outside any function, accessible throughout the program.
- **Local Variable:** Declared inside a function, accessible only within that function.

Example:

```
1 x = 10          #      Global Variable
2
3 def calculate():
4     x = 5        #      Local Variable (shadows global x)
5     y = 2
6     z = x + y    #      Uses local x
7     print("z =", z)
8
9 calculate()
10 print("x =", x) #      Uses global x
```

Note:

- Local and global variables can have the same name.
- Inside the function, Python uses the **local** variable by default.

Problem: If a local variable and global variable have the same name, the function by default refers to the local variable and ignores the global one.

Solution: Use the `global` keyword to access and modify the global variable inside a function.

Example:

```
1 x = 10
2
3 def update():
4     global x      # Accesses global variable
5     x = x + 5
6     print("Inside:", x)
7
8 update()
9 print("Outside:", x)
```

Explanation:

- Without the `global` keyword, a new local variable `x` would be created.
- With `global`, the original variable is modified.

- If a local variable and a global variable have the same name, then the function by default refers to the local variable and ignores the global variable.
- This means the global variable is not accessible inside the function, but it is accessible outside the function.
- In this situation, if we need to access the global variable inside the function, we can use the `global` keyword followed by the variable name.

Without using global

```
a = 50
def show():
    a = 10
    print(a)

show()
print("a:", a)
```

Using global

```
a = 50
def show():
    global a
    print(a)
    a = 20
    print(a)

show()
print("a:", a)
```

- The `globals()` function returns a table of current global variables in the form of a dictionary.
- It can be used to access a global variable even if a local variable has the same name.

Example

```
a = 50
def show():
    a = 10
    print("Local Variable A:", a)
    x = globals()['a']
    print("X:", x)

show()
print("Global Variable A:", a)
```

- In languages like C or Java, values are passed **by value** or **by reference**.
- In Python, arguments are passed using **object references**.
- This means:
 - The reference (address) of the object is passed to the function.
 - If the object is **mutable** (e.g., list, dict), changes inside the function affect the original object.
 - If the object is **immutable** (e.g., int, str, tuple), changes create a new object, leaving the original unchanged.

Example 1: Immutable Object (Integer)

Code

```
def val(x):  
    x = 15  
    print(x, id(x))  
  
x = 10  
val(x)  
print(x, id(x))
```

Explanation:

- Integers are **immutable**.
- When `x = 15` is executed, a **new object** is created in memory.
- The global `x` remains unchanged (10).

Example 2: Mutable Object (List)

Code

```
def val(lst):  
    lst.append(4)  
    print(lst, id(lst))  
  
lst = [1, 2, 3]  
print(lst, id(lst))  
val(lst)  
print(lst, id(lst))
```

Explanation:

- Lists are **mutable**.
- The same object reference is passed to the function.
- Adding a new element modifies the **same list** in memory.

What is Recursion?

- A function calling itself again and again.
- Used to solve problems by breaking them into smaller sub-problems.

Advantages

- Simplifies complex problems.
- Easier for sequence generation (e.g., factorial, Fibonacci).
- Code looks clean and short.

Disadvantages

- Uses more memory and time (expensive).
- Harder to debug.
- Sometimes tricky to understand.

Factorial: With and Without Recursion

With Recursion

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)  
  
print(factorial(5)) # 120
```

Without Recursion

```
def factorial(n):  
    result = 1  
    for i in range(1, n+1):  
        result *= i  
    return result  
  
print(factorial(5)) # 120
```

- Print the multiplication table of 12 using recursion.
- Write a function to calculate the area and perimeter of a rectangle.
- Write a function to calculate the area and circumference of a circle.
- Write a function to calculate the power of a number raised to another.
- Write a Python function to find the maximum of three numbers.
- Write a Python program to convert degrees to radians.

Expression / Expression Statements

- Used to compute and return a value, or to call a function.
- Examples: `2+3`, `a-b`, `print("Hello")`.

Statement

- Represents a complete instruction in Python.
- Can span one or more lines of code.
- Expressions are also statements.
- Examples: `if` statement, assignment (`x = 5`), loops.

Definition

- A function without a name is called an **Anonymous Function**.
- Also known as a **Lambda Function**.
- Defined using the `lambda` keyword (not `def`).

Syntax:

```
lambda argument_list : expression
```

Examples:

```
lambda x : print(x)  
lambda x, y : x + y
```

Creating a Lambda Function

Syntax:

```
lambda argument_list : expression
```

Example:

```
lambda x, y : x + y
```

Explanation:

- `lambda` → creates an anonymous function.
- `x, y` → function arguments.
- `:` → separates arguments from the expression.
- `x + y` → expression to be returned.

Examples:

```
# Lambda with one argument
sum = lambda x : x + 1
print(sum(5))      # Output: 6
```

```
# Lambda with two arguments
add = lambda x, y : x + y
print(add(5, 2))   # Output: 7
```

- **No Name:** A Lambda Function doesn't have any name.

```
lambda x : print(x)
```

- **Returns a Function:** A Lambda can be assigned to a variable.

```
show = lambda x : print(x)
```

- **Arguments:** A Lambda function can take zero or any number of arguments but contains only one expression.

```
lambda x, y : x + y
```

- **Return Statement:** No need to write return — the expression is returned automatically.
- **Body Restriction:** Can only contain expressions (not statements).
- **Arguments Support:** You can use all types of actual arguments (positional, default, variable-length, etc.).

Definition: When we write a lambda function inside another lambda function, it is called a **Nested Lambda Function**.

Example:

```
add = lambda x=10 : (lambda y : x + y)

a = add()           # returns inner lambda function
print(a)            # <function <lambda> ... >
print(a(20))        # Output: 30
```

Passing Lambda Function to another Function

Concept: We can pass a lambda function as an argument to another function.

Example:

```
def show(a):  
    print(a(8))  
  
# Passing lambda function as argument  
show(lambda x : x)      # Output: 8
```

Concept: A function can return a lambda function as its result.

Example:

```
def add():  
    y = 20  
    return (lambda x : x + y)  
  
a = add()           # 'a' now refers to the returned lambda  
print(a(10))        # Output: 30
```

Immediately Invoked Function Expressions (IIFE)

Concept: An IIFE is a function (here, a lambda) that is defined and called immediately without being stored in a variable.

Examples:

```
# Normal lambda usage
```

```
sum = lambda x : x + 1
```

```
print(sum(5))           # Output: 6
```

```
# IIFE with one argument
```

```
print((lambda x : x + 1)(5))  # Output: 6
```

```
# Normal lambda with two arguments
```

```
add = lambda x, y : x + y
```

```
print(add(5, 2))          # Output: 7
```

```
# IIFE with two arguments
```

```
print((lambda x, y : x + y)(5, 2))  # Output: 7
```

- A **Decorator** is a function that adds extra features to another function *without changing its code*.
- A decorator takes a function as input, wraps it inside another function, and returns the new function.
- We use the symbol `@decorator_name` before the function definition to apply a decorator.

Key Idea: Decorators let us **reuse code** (like logging, checking permissions, measuring execution time) by wrapping existing functions.



Placeholder Citation 1, "Title of Paper 1," in *Placeholder Journal Name*, Year, pp. X-Y.



Placeholder Citation 2, "Title of Paper 2," in *Placeholder Journal Name*, Year, pp. X-Y.